

My starting point for this test task was basically an OpenAI subscription and nothing else. Because of that, the code only uses Codex calls. This means that I am building specialized agents on top of a general one, which makes progress harder to show: the baseline is already one or several Codex runs, and Codex can already do a lot.

For the base of the architecture, I chose a JSON file that stores the structure of model calls and one model with persistent memory. I called this model the architect. The idea was to put it between the agents that perform specialized parts of the task and the final goal, so that only the architect builds the call graph.

To implement this, I decided to write a Python library that calls Codex in a loop by itself, so that the process has fewer places where it can break.

I decided to test everything on the prompt “create Tetris in the browser”, because it is simple, basic, and easy to test.

When I finally set everything up, the first agent run failed. The architect completed the whole task by itself and wrote Tetris. At first, I thought the problem was that the architect did not understand what it was supposed to do and that I needed to improve the prompt. But after looking more closely, I found that Codex did not have permission to write into service directories, while I was storing the graph in `.agents/`. A simple solution would have been to make the architect output the graph as the result of the program and then have the Python loop save it into the right folder. But that makes the system more complicated and increases the chance of bugs when passing the graph around. So I rewrote the permission config for Codex runs.

On the next run, the loop started. It created a programmer and a tester, but the tester could not run the tests because of missing permissions and said something like: “the code looks correct, I could not run the tests, everything is fine.” After that, I added a rule for the architect: if it sees an error it cannot fix, it must stop with an error and explain the reason. Separate agents, if they run into problems, must call the architect. With this change, I debugged the code to a working state very quickly.

At that point, my system was generating silly cycles like programmer→tester→end. Usually, the programmer did all the hard parts.

```
(.venv) ilyakondakov@ilyas-MacBook-Air tetris % stem-agent graph --debug-log .agents/de

[architect] done 42s events=10 output=2705 total=111967
[node:browser_tetris_programmer] done 256s events=46 output=16527 total=452515
[node:browser_tetris_verifier] done 314s events=56 output=20967 total=748605
[architect] done 36s events=6 output=4595 total=182466
result={"checks_run":["node --check script.js","Node VM smoke test loading script.js with
```

Figure 1: A simple programmer→tester→end cycle.

Even with just this verification step, Tetris started working better. I tried to generate Tetris three times: two times through a single Codex run, and one time through my system.

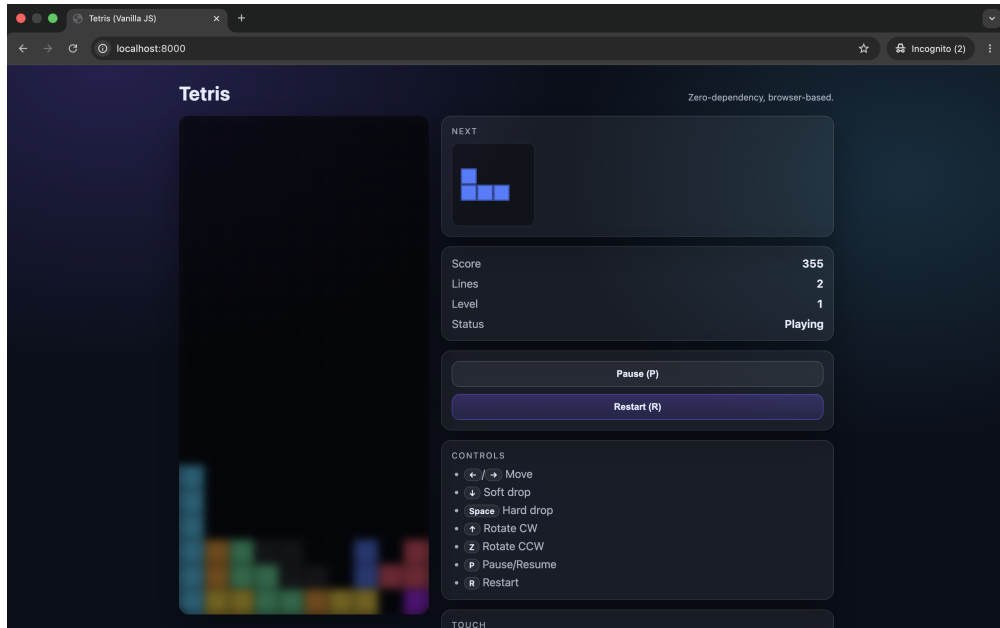


Figure 2: Version by Codex; for some reason, the field is blurred.

One of the versions crashed when a block fell, and in the second version the screen was blurry. I do not know how I got that unlucky. In theory, making Tetris from one prompt should be easy, but in the end only the stem agent handled it.

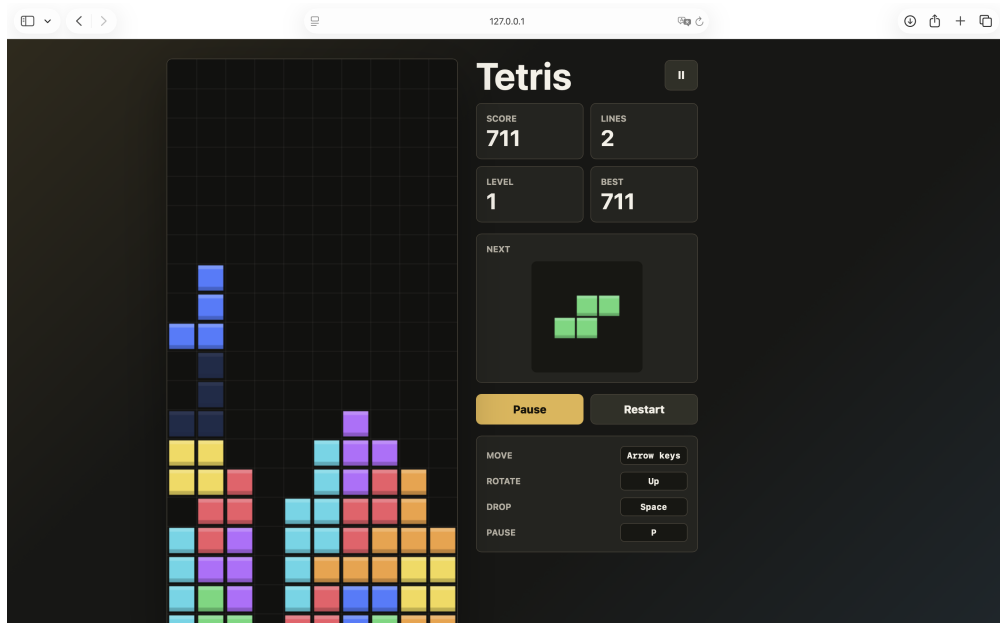


Figure 3: Tetris by the stem agent.

To fix the problem that the architect almost always generated the graph programmer→tester and did not really use specialization, I simply changed the prompt. I asked it to create more

nodes and specialize tasks better. After that, the trees became more complex, and even cycles started to appear. I think this is the strongest part of the current architecture: almost all future changes can be made only through the general prompt. For example, I can describe in more detail, with examples, how to specialize the graph for research tasks, code-writing tasks, and so on, or I can tell it to search for the best architecture for the task in the internet.

At this point, I had a mostly working stem agent, and the question became how to test it with a measurable metric. I remembered that I had recently taken part in the selection round for the Midnight Code Cup. The tasks there had the form: come up with an example that maximizes X. There is no single best answer, and the solution can be improved almost indefinitely, which makes it a good metric for comparing models. That is why, in my GitHub repository, you will find the branches `taskCursor` and `taskMaze`, with task statements and validators used to measure the score.

I used these tasks to compare agents, but the process was not very smooth. For example, in `taskMaze`, the statement guarantees that mazes are generated randomly by a generator. As examples, it gives mazes with seeds from 1 to 10. For some reason, the researcher decided that the brilliant idea was to hard-code the tests from the examples. The solution validator said that the idea was bad, because the seed could be different. BUT AFTER THAT THE RESEARCHER DECIDED TO HARD-CODE NOT 10, BUT 50 SEEDS, and stopped there. Honestly, I still do not understand how that happened. After that, I changed the validator so that it generated seeds and tests in real time, and in the new loop the stem agent wrote a normal solution.

The final result was 606.2/800 on `taskCursor` and 435.6/800 on `taskMaze`. A score of 800 was given for the best solution in the round. Our team got around 200 on `taskCursor`, even though we used models during the process. So it is up to you to decide whether we messed up during the round, whether GPT-5.5 is that much better than GPT-5.4, or whether the stem agent actually works.

For comparison, a single Codex run scores 113.8/800 on `taskMaze` and 427.7/800 on `taskCursor`.

One could say that my agent simply consumes many more tokens and therefore gets a better result. I will not argue with that. I will only note two things. First, Codex is launched in `xhigh` mode, so it is already expected to spend as many tokens as it needs, and stopping is its own decision. Second, in the Tetris example, Codex somehow produces unfinished products in a normal run, without testing. So actually completing the task matters more.

Further improvements are mostly prompt changes and quality-of-life features, such as tracking token usage and asking the agent to stop when it gets close to the limit. In the current pipeline, this is hard to do because Codex reports how many tokens it used only when it gives the final answer, which can happen after up to half an hour of reasoning. There is a funny detail related to this: I did not know what to show the user so that it would be clear that the agent was still working, so I started printing a live timer. It really looks much better than showing nothing.